

DESCRIPTION

owl (rigorously in lower letters) is both a small language based on character interpretation, in the great galaxy of obfuscated languages and ain interpreter of this language. It is based upon forth, the language created by C. Moore, and "false", an obfuscated language (the ancestor of all character-based languages) by Wouter van Oortmerssen. Therefore thanks to Charles and Wouter.

SYNOPSIS

```
owl <options> <filename> <commands>
```

PARAMETERS

<options> are:

```

--conditions print redistribution conditions from GPL3
-d           execute program in debug mode
-D           in debug mode, print var contents
-e           toggle PAD's erasure after string initializations
-F           if in debug mode, print functions buffers contents
-h, --help   display this information
-i           toggle integer division mode (default: standard)
-p '<string>' parse <string> enclosed in quotes and exit
-r           toggle rounding mode (default: cut)
-S           if in debug mode, enable single-step running
-t           print a timer report after execution
-v, --version display version, license and warranty
--warranty   print warranty conditions from GPL3

```

<filename> is a source text file containing owl code lines to be executed. Usually, such sources have extension ".owl", but this can be omitted in <filename>.

Option -p <string> is used to execute portions of code (contained into <string>) without a file to process. If no filename is given or option -p is not used, owl will complain.

Code contained into the facultative <commands> will be executed before program start (unless option -p is used, in which case <commands> are appended to <string>).

In <commands> and <string> all characters that have a special meaning for the shell you use should be escaped, and double/single quotes should be used.

CONVENTIONS

ENVIRONMENT

The environment in which owl operates is composed by:

- * an integer stack; storable values are 64-bits integer (if used as 8-bits characters, only the lower 8 bit are used, signed or unsigned depending on the contest). The STACKLIMIT constant in dib.h defines the depth of the stack (by default set to 1024).

- * a 2-cells function buffer, to store two functions, thought as sets of instructions and data; this buffer works as a two-element stack, actually, but it differs from a stack because any command using one or both functions erases the buffer after finishing its task. The length of each function LIMIT (by default set to 1024).

- * an array of chars, called PAD, holding only 8-bits integers, used as a string storage; any command using a string, sets/gets the string

from PAD; every string loaded and printed by owl from the source code is primarily hosted on PAD; every command that deals with characters and strings, use the unsigned 8-bits pattern in the range 0÷255. Every command that deals with numbers use the signed 8-bits pattern in the range -128÷127. The length of the PAD is LIMIT (by default set to 1024).

* an array of integers, called simply array, holding true 64-bits signed integers; it's organized as a long unidimensional array; it's up to the programmer to write code for using chunks of it, to hold many arrays, or one or more multidimensional arrays (matrices), provided memory is enough. The length of the array is ARRAYLIM (by default set to 32768).

Limits constants may be changed at need in the owl.h source file before compiling.

ELEMENTS OF THE LANGUAGE

n	a signed 64-bits integer
m	an unsigned 32-bits integer for system constants
c	an unsigned 8-bits integer
b	a boolean flag (a true/false integer)
[f]	a function in the functions buffer
x*n, y*n	any stack configurations
<stream>	the input typed by the user (any input request)
'stream'	the code following/preceding the command (no blanks)
"stream"	a string in PAD (any content)
v	a lowercase character a÷z for function variables
V	an uppercase character A÷Z for integer variables

STACK NOTATION

All commands are exposed with their stack notation; the stack notation is a heritage of the forth language and its purpose is to depict the net stack situation before and after the command execution; for instance, if a (fake) command c takes two values from the stack (in order, a boolean and an integer), and returns two other integer values, the stack notation is:

(n1 b - n2 n3)

The dash separates the stack before (left) from the stack after (right). b is on Top of Stack, n1 just under. n2 is returned first, n3 then, and stays on Top. If no values are needed by c but some values are returned, the stack notation is:

(- n1 n2)

If values are needed but no values are returned, the stack notation is:

(n1 n2 -)

If no values are needed nor returned, the stack notation (while unnecessary) is:

(-)

Numerical suffixes identify different elements of the same class: e.g. n1 and n2 are both integers and are different entities (even if they may yield the same value), while n1 and m1 are different variable types even if they share the same suffix.

Special symbols have been introduced in the proper owl's notation:

"stream" is a string in the PAD; the command, in order for a correct execution, must find a proper string in PAD

'stream' is the character immediately after or before in the parsing line, and the command must be 'attached' to it

<stream> is the user typing in request of an input or the data read from file; in general it is placed in the 'stack after' section.

[f] is a function in the function buffer; commands using functions look in the function buffer, load and use them, erasing buffers afterwards

Finally, at times a command may work in two ways, using alternative inputs or outputs; in these cases the symbol | separates the two groups:

Alternative input

```
( "stream" 'v' | "stream" n - b )
```

Alternative output

```
( n1 m1 - n2 m2 | b )
```

THE LANGUAGE STATEMENTS

Commands dealing with programming features are:

LANGUAGE COMMANDS

```
!      ( [f1] [f2] - x*n ) while [f1] execute [f2]
```

```
!      ( [f] - x*n ) repeat executing [f] until true
```

```
! ?    ( ? - ) exit immediately
```

```
"      ( - ) start a string, until next not escaped "
```

```
"      ( - "stream" ) end a string, store to PAD and print it
```

```
""     ( - "stream" ) end a string and store to PAD without print
```

```
#      ( - ) start a comment anywhere, until end of line
```

```
#,     ( n m - ) store n at position m of int array
```

```
#@     ( m - n ) fetch n from position m of int array
```

```
$      ( n1 n2 - n2 n1 ) swap
```

```
%      ( n - n n ) dup
```

```
&      ( n1 n2 - n3 ) bitwise and
```

```
'      ( x*n m - y*n ) roll mth element
```

```
(      ( <stream> - c ) get char c from keyboard as ASCII, no echo
```

```
(*     ( - ) start a multi-line comment, until *)
```

```

)      ( c - ) emit c as ASCII char to terminal
*      ( n1 n2 - n1*n2 ) multiplication
*)     ( - ) end a multi-line comment
+      ( n1 n2 - n1+n2 ) addition
,      ( [f] 'v' - ) store last function in variable v (v,)
,      ( n 'V' - ) store n in variable V (V,)
,      ( c m - ) put char c to position m of PAD
,,     ( 'v' | n - "stream" ) copy function v/n to PAD
-      ( n1 n2 - n1-n2 ) subtraction
.      ( n - ) print n as number to terminal
/      ( n1 n2 - n1/n2 ) integer division
:      ( n1 n2 - n3 ) leave n3 = n2th root of n1
;      ( n - ) drop
<      ( <stream> - n ) read a number from keyboard
<<     ( n1 n2 - n3 ) n3 = n1 shifted left n2 bits (increasing)
=      ( n1 n2 - b ) true if n1 = n2
>      ( n1 n2 - b ) true if n1 > n2
>>     ( n1 n2 - n3 ) n3 = n1 shifted right n2 bits (reducing)
?      ( n [f1] [f2] - x*n ) if n is true, execute [f1], else [f2]
?      ( n [f] - x*n ) if n is true, execute [f]
?!     ( ? - ) exit immediately
@      ( 'v' - x*n ) execute function v (v@)
@      ( 'V' - n ) fetch value from var V (V@)
@      ( m - c ) fetch c from pos m of PAD
@,     ( 'v' | n - ) set function index to function v/n
@@     ( - x*n ) execute function pointed by function index
[      ( - ) start a function until last ]
\      ( n - -n ) negate TOS
]      ( - [f] ) close a function and put it in the function buffer
]      ( 'stream' - c ) leave next character in stream as ASCII c

```

`^` (`n1 n2 - n3`) leave `n3 = n1` to the power of `n2`
`_q` (`- m`) push stack depth (quantity)
`_'` (`"stream" 'v' | "stream" n -`) put PAD in var `v/n` as string
`_,` (`"stream" 'v' | "stream" n -`) put PAD in var `v/n` as function
`_<` (`"stream" m1 - n m2 | 0`) convert number `n` in PAD from pos `m1`
`_=` (`'v1' 'v2' | n1 n2 - n`) compare functions `v1/v2` or `n1/n2`
`_@` (`"stream" - x*n`) execute PAD as function
`_[` (`"stream" - x*n`) execute module (name between `_[` and `]`)
`_]` (`"stream" - x*n`) include file (name between `_]` and `[`)
`_~` (`"stream" 'v' | "stream" n1 - n2`) check `v` as substring of PAD
``` ( `x*n m - x*n n` ) pick `m`th element from stack  
`{` ( `<stream> - "stream"` ) read a string from keyboard to PAD  
`|` ( `n1 n2 - n3` ) bitwise or  
`}` ( `"stream" -` ) print PAD to terminal  
`~` ( `b - not b` ) logical not

The token `"` alone is equivalent to `"\n"`.

## FILES ARCHITECTURE

Commands dealing with file I/O are:

`_(<` ( `<stream> - c` ) get character `c` from read-file as ASCII  
`_)` ( `c -` ) emit `c` as ASCII char to write-file  
`_+` ( `-` ) toggle retain terminator in read-file lines  
`_.` ( `n -` ) print `n` as number to write-file  
`_C` ( `| n - b` ) close read- or write-file  
`_E` ( `- b` ) erase current open write-file  
`_F` ( `- b` ) `b` is true if EOF on read-file is reached  
`_O` ( `"stream" | "stream" n - b` ) open read- or write-file  
`_R` ( `| n - b` ) restore write-file position  
`_S` ( `| n - b` ) set seek read-file position  
`_U` ( `-` ) return last character read back to read-file  
`_{'` ( `<stream> - "stream"` ) read a string from write-file to PAD  
`_}'` ( `"stream" -` ) print PAD to write-file

Open/close flags are (\_O and \_C):

- 0 = read-file
- 1 = write-file

Restore flags are (\_R and \_S):

- 0 = restore file pointer to start of file
- 1 = restore file pointer to end of file

#### TRUTH VALUES

Truth values left by test operators are:

- 1 (all bits set) = true truth value
- 0 (all bits cleared) = false truth value

The following is also true: zero is false, not zero is true.

#### UNDERSCORE COMMANDS

Underscore commands are system-related commands:

- \_& ( - ) toggle ampersand to hex, B to binary, O to octal prefix
- \_b ( - ) set binary output
- \_c ( - | - n ) second call leaves milliseconds since first
- \_d ( - ) set decimal output
- \_e ( - ) clear both PAD and array
- \_h ( - ) set hexadecimal output
- \_i ( - ) toggle integer division mode (default: standard)
- \_o ( - ) set octal output
- \_r ( - ) toggle rounding mode (default: cut)
- \_s ( - ) send PAD's content to shell
- \_t ( - 6\*m ) push year, month, day, sec, min, hour
- \_td ( - m ) push day
- \_th ( - m ) push hour
- \_tm ( - m ) push minutes
- \_tn ( - m ) push milliseconds
- \_ts ( - m ) push seconds
- \_ty ( - m ) push year
- \_tM ( - m ) push month
- \_v ( - 3\*m ) push version
- \_x ( - ) set hexadecimal output

\_A    ( - m ) push limit of integer array  
 \_B    ( m1 - | m2 ) set numerical base to m1 or leave base if m1=0  
 \_D    ( - ) toggle debug mode  
 \_OS   ( - m ) push OS type (0 = UNIX, 1 = win32)  
 \_P    ( - m ) push limit of PAD

The character \_ alone is a no-op.

## ASCII

owl uses only ASCII characters for command tokens and operators, so that any program may be well written and read by any keyboard (arguably), and can print (EMIT, in forth words) characters in the whole 0-255 range; beware, though: this may not be portable, since different Operating Systems use different layouts for the ASCII table in the range 128-255, so that different Operating Systems may produce different outputs, or even be unable to treat those characters.

## ESCAPES

What follows is a table with escape chars understood by owl into strings (based on the ASCII / ISO 8859-1 (Latin-1) Table:

\0	(ASCII 00), end-of-string
\b	(ASCII 08), backspace
\t	(ASCII 09), horizontal tab space
\n	(ASCII 10), new line
\v	(ASCII 11), vertical tab
\f	(ASCII 12), form feed
\r	(ASCII 13), carriage return
\"	(ASCII 34), double quote
\'	(ASCII 39), single quote
\?	(ASCII 63), question mark
\\	(ASCII 92), backslash
\!	(ASCII 161), inverted exclamation mark
\c	(ASCII 162), cent
\L	(ASCII 163), pound Sterling
\q	(ASCII 164), barred circle
\Y	(ASCII 165), Japanese Yen
\	(ASCII 166), dashed vertical bar
\S	(ASCII 167), paragraph
\p	(ASCII 168), double superscript points
\C	(ASCII 169), copyright mark
\<	(ASCII 171), left angle quote
\m	(ASCII 172), logic 'not' symbol
\R	(ASCII 174), registered mark
\s	(ASCII 175), upperscore
\o	(ASCII 176), degrees sign
\+	(ASCII 177), plus-minus
\2	(ASCII 178), two superscript
\3	(ASCII 179), three superscript
\M	(ASCII 180), inverted tick
\u	(ASCII 181), Greek mi
\x	(ASCII 183), middle dot
\1	(ASCII 185), superscript one
\d	(ASCII 186), underlined degrees sign
\>	(ASCII 187), right angle quote
\I	(ASCII 191), inverted question mark

```

\A (ASCII 198), uppercase Latin AE
\p (ASCII 215), times
\B (ASCII 223), s-z German ligature
\a (ASCII 230), lowercase Latin ae
\− (ASCII 247), interval division
\O (ASCII 248), small Greek phi
\T (ASCII 254), Icelandic thorn

```

#### EXAMPLES

1. print numbers and their squares from 1 to 10, each pair on one line

```
1[%10>~][%."\"t"%%*."\"1+]!
```

2. input a string and print it

```
"Input your name >\"{ }", what's up?"
```

3. calculate factorial of number on TOS

```
"Factorial of \"%.\" is \"%1-[%1=~][%2'*$1-]!;
```

4. print all ASCII characters from 32 to 255 in a five-column table

```
32A,255Z,5T,T@1-K,[1-[1`1`10$^/~][\"\"t\"1-]!;.]p,
A@[%Z@>~][%%3p@9))1+K@[9)K@1-K,][10)T@1-K,][?]!;
```

5. cat simulator (file name should be on PAD)

```
"data.txt\"0_0[_+[_{ }_F]!][]" could not be opened."10)]?
```

6. print current hour and date in (UE format)

```
[":"]c,["-"]d,_t.c@.c@." "$.d@.d@.
```

#### SEE ALSO

The whole documentation is included as a textual file in the source package and summed up in this man/info page.  
 For bug reporting, please write to <ing dot antonio dot maschio at gmail dot com>.  
 It's GPL! Enjoy!

<ing.antonio.maschio@gmail.com>" owl 0.7.8

owl(1)